Making SQL Query Optimization Explainable

1st Manuela Guédez
Dept. de Informática e IA
UCU
Montevideo, Uruguay
manuela.guedezl@ucu.edu.uy

2nd Guillermo Tula
Dept. de Informática e IA
UCU
Montevideo, Uruguay
guillermo.tula@correo.ucu.edu.uy

3rd Bernardo Rychtenberg
Dept. de Informática e IA
UCU
Montevideo, Uruguay
brychtenberg@ucu.edu.uy

4th Fiorella Cravero
Dept. de Informática e IA
UCU
Montevideo, Uruguay
fiorella.cravero@ucu.edu.uy

***Abstract*—Cost-based SQL query optimizers select execution plans from a large combinatorial search space, yet the factors driving these decisions remain largely opaque. This paper presents an Explainable Artificial Intelligence (XAI) framework that improves the transparency of SQL query optimization through feature attribution. Candidate execution plans are vectorized into interpretable plan-level features and a local proxy model is trained per query, enabling SHAP to quantify the contribution of each feature to the optimizer's estimated cost. The framework produces both global importance rankings across the full workload and local waterfall explanations for individual queries, complemented by natural-language summaries that relate mathematical attributions to structural SQL concepts. Experiments on a controlled synthetic workload contrast a classical Selinger-style optimizer against a Bayesian cardinality estimation approach, revealing distinct optimization patterns: Selinger distributes cost sensitivity across multiple structural features, while the Bayesian optimizer concentrates it on initial filter selectivity. Runtime validation via q-error confirms that explanations remain structurally faithful to the cost model while simultaneously exposing cardinality estimation weaknesses. These results demonstrate the potential of XAI as a diagnostic and analysis tool for understanding, debugging, and ultimately improving database query optimizers. Future work will extend the evaluation to real-world benchmarks and incorporate additional explainability techniques.**

***Index Terms*—SQL, XAI, Query Optimization.**

I. Introduction

The continuous growth in data volume and query complexity has made efficient query processing a central requirement of modern database systems, with query optimizers responsible for selecting, among a combinatorial space of semantically equivalent execution plans, the one expected to minimize resource consumption and/or response time. This problem has been central to relational database engineering since the seminal work of System R [1], which established the paradigm of searching over alternative join orders and access paths guided by cost estimation. Even for relatively simple queries, the number of alternative plans arising from different join orders, access methods, and operator implementations grows in a combinatory way, so modern optimizers cannot exhaustively evaluate every alternative. Instead, they rely on heuristics, pruning strategies, and cost estimation models [2], [3]. However, cardinality estimation errors remain a primary cause of suboptimal query plans [4], leading to unpredictable performance degradation [5].

To improve accuracy, modern database research has explored probabilistic approaches [6] and machine learning models for cardinality estimation [7] and query optimization [8]. Despite promising results, evaluating the real-world readiness of these learned models is challenging [9]. Recent frameworks such as BayesCard [10] revitalize cardinality estimation through probabilistic graphical models trained on the underlying data, offering a balance between accuracy and structural transparency.

While these approaches enable practical optimization times, they also make optimizer decisions difficult to understand and interpret. Database administrators (DBAs) and researchers often have limited visibility into why a particular plan was selected over competing alternatives, or which query and data characteristics had the greatest influence on that decision. This opacity persists even for expert developers, a problem exacerbated by the increasing use of complex machine learning components in databases.

Recent advances in Explainable Artificial Intelligence (XAI) provide promising tools for addressing this limitation [11]. While methods like LIME [12] provide local approximations, feature attribution methods based on game theory offer consistent insights into the underlying decision-making process. This work investigates the application of SHapley Additive exPlanations (SHAP) [13] to SQL query optimization. The central hypothesis is that understanding which query and data characteristics most strongly influence an optimizer's cost-based decisions can improve transparency, facilitate debugging and analysis of optimizer behavior, and ultimately inform the design of more interpretable query optimization strategies.

II. Materials and Methods

Fig. 1 presents the overall architecture of the proposed explainability framework, illustrating the relationship between the query processing pipeline, the feature attribution module, and the runtime validation step.

A. Query Processing and Optimizers

Query processing in this work refers to the complete workflow that transforms an input SQL query into a set of candidate execution plans. Each query is first parsed and represented as an Abstract Syntax Tree (AST), preserving the logical semantics of the query while enabling the system to explore alternative physical strategies for obtaining the same

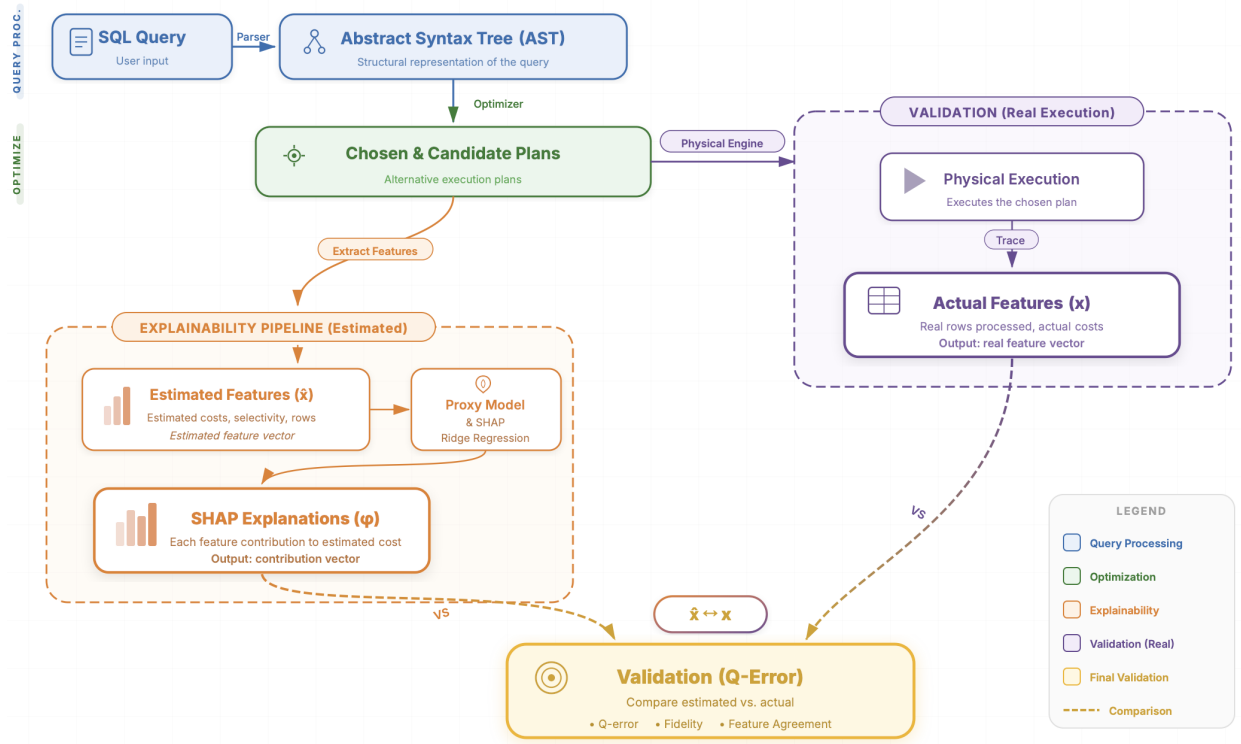


Fig. 1. Architecture of the explainability framework. The query processing pipeline (blue/green) branches into the explainability pipeline, which extracts estimated features and computes SHAP attributions, and the real execution path, which collects actual runtime features. Both outputs are compared via Q-error to validate explanation fidelity.

result. The optimizer explores alternative access paths and join orders, estimates the cost of each candidate plan using available statistics, and selects the plan expected to minimize execution work.

Two cost-based optimization paradigms are considered in this study to evaluate the explainability pipeline across different search spaces (while other modern approaches like deep reinforcement learning for join enumeration [14] present different XAI challenges, they are outside the scope of this work):

- *Classical Selinger-style Baseline*: A classical cost-based optimizer [1] that estimates the cost of alternative join orders using basic statistics and dynamic programming. It exhaustively permutes tables in the join sequence, providing a broad search space where intermediate selectivities and cardinalities heavily drive the plan choice.
- *Bayesian Cardinality (BayesCard)*: Adapted from the BayesCard framework [10], this optimizer employs Bayesian Networks for cardinality estimation. Unlike the baseline, it keeps the initial table fixed to reduce the search space and relies on complex probabilistic relationships to estimate selectivities and filter rows. It exposes intermediate quantities that can be analyzed using feature attribution methods.

B. Plan Vectorization

The optimizer operates over an AST representation of each candidate plan. Although the AST fully describes the optimizer's decision process, it cannot be directly used by numerical feature attribution methods such as SHAP. To bridge this gap, each candidate plan undergoes a process called *plan vectorization*, where the optimized AST is mapped to a structured, numerical feature vector that summarizes the characteristics relevant to cost-based optimization.

For each candidate plan, the extracted feature vector includes structural characteristics (e.g., table and predicate counts) and estimated quantities, including initial table cardinalities (`base_filtered_rows`), predicate and join selectivities, intermediate result sizes after each join (`join_N_output_rows`), and accumulated estimated work (`sum_intermediate_rows`). These variables, summarized in Table I, constitute the vocabulary through which optimizer decisions are explained.

TABLE I
MAIN PLAN-LEVEL FEATURES USED FOR EXPLANATION

Feature	Description
`table_count`	Number of tables in the query plan
`predicate_count`	Number of predicates in the query
`base_filtered_rows`	Estimated rows after base-table filtering
`join_N_table_rows`	Estimated rows of the table added at join step N
`join_N_selectivity`	Estimated selectivity of join step N
`join_N_output_rows`	Estimated intermediate rows after join step N
`sum_intermediate_rows`	Accumulated estimated intermediate rows
`estimated_cost`	Estimated plan cost

C. Explainability Pipeline

The explainability process starts by generating alternative candidate plans for each SQL query corresponding to different join orders. The XAI module records the selected plan and the set of runner-up alternatives.

Feature Attribution: Feature attribution is performed using SHAP [13], which assigns a contribution value to each feature according to its influence on the optimizer's estimated cost. A proxy model (a lightweight linear Ridge regression model) is trained exclusively on the candidate plans generated for the specific query under analysis to approximate the optimizer's cost estimates. This preserves the local nature of the explanations, reflecting the factors that most influenced the decision for an individual query rather than a global description.

Structured and Comparative Explanations: Beyond raw SHAP values, the pipeline generates structured, natural-language explanations to facilitate human interpretation. This includes an executive summary highlighting the primary cost drivers (positive and negative factors) and a *comparative explanation* benchmarking the selected plan against the runner-up plan (the closest alternative). By explicitly comparing the selected plan against its closest competitor, the pipeline clarifies why one sequence of intermediate joins was preferred over another.

III. Experimental Setup

A. Synthetic Database and Workload

A synthetic database was designed to support a controlled explainability analysis. The schema consists of relational tables containing integer and categorical attributes, populated to follow predefined cardinalities and value distributions. This controlled environment exposes meaningful optimization trade-offs and ensures that plan selection depends on measurable query and data characteristics. A batch process generated multiple SQL join queries with varying predicates, creating a rich dataset of optimization instances.

B. Metrics

To evaluate both the fidelity of the generated explanations and the global behavior of the query optimizers, we employ two complementary metrics. The first metric assesses the alignment between the estimated database statistics (which serve as the basis for explainability) and actual execution values. The second metric aggregates individual explanation weights to quantify which database features play the most critical role across the entire query workload. These metrics are detailed below:

- *Q-Error*: To assess whether the explanatory features faithfully represent runtime behavior, the database engine executes the optimized AST and collects a runtime execution trace. The discrepancy between estimated feature values ($\hat{x}$) and runtime-derived observations (x) is measured using *q-error*:

$$q\text{-error} = \max\left(\frac{\hat{x}}{x}, \frac{x}{\hat{x}}\right) \tag{1}$$

  A q-error of 1 indicates perfect agreement. This metric is used to validate the fidelity of the explanations: an explanation is considered robust if the estimated features that heavily influenced the plan choice (e.g., intermediate result sizes) closely match the actual quantities observed during query execution.
- *Global feature importance*: Global feature importance was calculated as the mean absolute SHAP contribution of each feature across all candidate plans:

$$I_j = \frac{1}{N}\sum_{i=1}^{N} |\phi_{ij}| \tag{2}$$

  where ϕ_{ij} is the SHAP contribution of feature j for candidate plan i. The results were then normalized as:

$$I_j^{(\%)} = \frac{I_j}{\sum_{k=1}^{m} I_k} \times 100. \tag{3}$$

IV. Results and Discussion

The workload consisted of 100 SQL queries evaluated with both the Selinger-style and Bayesian optimizers, resulting in 200 successful optimization runs. Selinger evaluated an average of 3.53 candidate plans per query, while Bayes evaluated 1.47. This aligns with their respective search space strategies, as Bayes fixes the initial table.

A. Global Feature Attribution

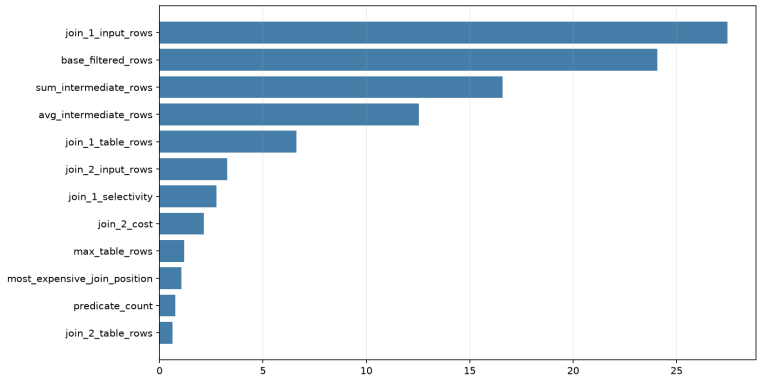
Fig. 2(a) shows that Selinger produced a distributed importance profile. The most influential features were related to the input of the first join, the filtered rows of the initial table, and the accumulated intermediate rows. This is consistent with its larger search space, as it evaluates different initial tables, making the plan cost sensitive to multiple structural features.

In contrast, Fig. 2(b) demonstrates that `base_filtered_rows` is the dominant feature for the Bayesian optimizer. Because Bayes keeps the initial table fixed, the variation in cost across candidate plans is heavily constrained by the initial filter selectivity. This feature represents the estimated number of rows entering the subsequent join sequence, strongly anchoring the total estimated cost.

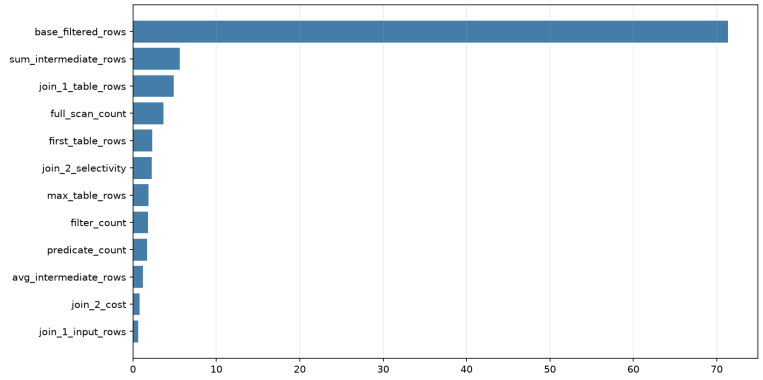
B. Local and Comparative Explanations

To understand individual query decisions, the framework generates local explanations using SHAP waterfall plots (Fig. 3). These plots decompose the optimizer's final cost estimate ($f(x)$) for a specific plan, starting from the expected base cost ($E[f(x)]$) across all plans. Each row represents a feature, and the length and direction of the bar quantify how that specific feature pushes the cost higher (red, pointing right) or lower (blue, pointing left) relative to the baseline.

For better interpretation of these explanations, a Large Language Model (LLM) is incorporated as a communication layer. The LLM receives the structured information extracted from the SHAP explanation and generates the following output:



(a) Selinger-style optimizer



(b) Bayesian optimizer

Fig. 2. Global feature importance comparison between the (a) Selinger-style and (b) Bayesian optimizers.

Selinger operates as a complex balancing act, weighing multiple intermediate steps and base selectivities to navigate a broad search space. In contrast, the Bayesian optimizer's visual footprint is overwhelmingly dominated by a single feature, `base_filtered_rows`, indicating a highly constrained, greedy strategy that heavily anchors its entire cost estimation on identifying the most restrictive initial filter. Ultimately, these visual explanations distinguish between a broad, trade-off-driven search and a narrow, selectivity-driven one, without ever examining their source code.

Fig. 3(a) demonstrates this dynamic for the Selinger-style optimizer. In this example, the penalty of a suboptimal join order is explicitly attributed to positive (red) contributions from factors like `max_intermediate_rows` and `most_expensive_join_cost`, which in combination drive the cost increase. In contrast, Fig. 3(b) illustrates a decision by the Bayesian optimizer, where the structural efficiency of the plan yields universally negative (blue) SHAP values. Features capturing early selective predicates, such as `sum_intermediate_rows`, act in concert with others to pull the projected cost down, with no structural penalties observed.

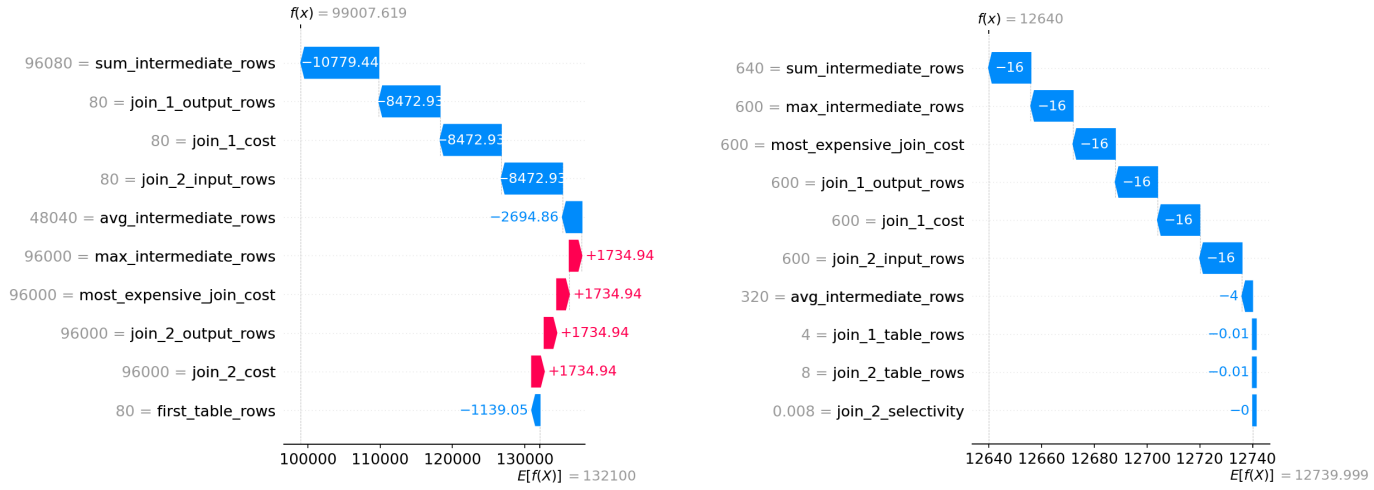
By combining SHAP visualizations with automatically generated natural-language summaries, the framework provides both quantitative and human-readable explanations of optimizer decisions. This enables DBAs to quickly identify which plan characteristics contributed most to the estimated cost, facilitating query-plan analysis without requiring direct inspection of the underlying feature attributions.

C. Estimated vs. Runtime Validation

Runtime tracing revealed that while the estimated intermediate cardinalities drove the optimizer's decision, they did not always perfectly match runtime observations. However, the *relative* magnitudes often held true. The q-error evaluation showed that explanations remain structurally faithful to the optimizer's cost model (the XAI correctly explains *why* the optimizer made a choice). When the q-error diverges significantly from 1, the explanation serves a dual purpose: it explains the optimizer's logic while simultaneously exposing weaknesses in the cost model's cardinality estimation, acting as a diagnostic tool for query engine developers.

V. Conclusions

This work presented an exploratory framework for improving the transparency of cost-based SQL query optimization through XAI techniques. By vectorizing execution plans into interpretable numerical features and applying SHAP-based feature attribution via a local proxy model, the proposed



(a) Selinger-style optimizer (b) Bayesian optimizer

Fig. 3. Local SHAP feature attribution (waterfall plots) for a selected plan. (a) Highlights the penalization of a suboptimal join order driven by intermediate rows. (b) Shows cost reduction driven by early filter selectivity.

approach quantifies the contribution of each plan characteristic to the optimizer's estimated cost, producing both global and local explanations.

The experimental results demonstrate that the framework effectively distinguishes between optimization strategies. The classical Selinger-style optimizer exhibits a distributed importance profile, with cost sensitivity spread across multiple structural features such as initial join inputs, filtered base rows, and accumulated intermediate cardinalities. In contrast, the Bayesian optimizer concentrates its cost sensitivity on the initial filter selectivity (`base_filtered_rows`), consistent with its constrained search space that fixes the leading table. These distinct attribution patterns provide interpretable evidence of how each optimizer navigates its decision space, offering insights beyond traditional performance metrics. These results suggest that feature attribution captures characteristic optimization behaviors rather than merely explaining isolated predictions.

Runtime validation through q-error analysis confirms that the generated explanations remain structurally faithful to the optimizer's internal cost model. When significant discrepancies arise between estimated and observed feature values,the framework serves a dual diagnostic purpose: it explains the optimizer's reasoning while simultaneously exposing weaknesses in its cardinality estimation, thereby functioning as a debugging tool for query engine developers.

Given the exploratory nature of this study, the evaluation was conducted on a synthetic database designed to isolate optimizer behavior in a controlled setting. While this approach enables precise analysis of the framework's capabilities, it does not capture the full complexity of real-world query workloads. Future work will extend the evaluation to established benchmarks such as JOB and TPC-H,incorporate additional XAI techniques including LIME and Integrated Gradients, and investigate metrics for systematically assessing the fidelity and utility of the generated explanations. Ultimately, explainability is envisioned as a complementary tool for the analysis, debugging, and design of increasingly sophisticated query optimization systems.

References

[1] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, "Access path selection in a relational database management system," in *Proc. 1979 ACM SIGMOD Int. Conf. Management of Data*, 1979, pp. 23–34.

[2] G. Graefe, "Query evaluation techniques for large databases," *ACM Computing Surveys*, vol. 25, no. 2, pp. 73–169, 1993.

[3] H. Lan, Z. Bao, and Y. Peng, "A survey on advancing the DBMS query optimizer: Cardinality estimation, cost model, and plan enumeration," *Data Science and Engineering*, vol. 6, no. 1, pp. 86–101, 2021.

[4] V. Leis, A. Gubichev, A. Kemper, and T. Neumann, "How good are query optimizers, really?" *Proc. VLDB Endowment*, vol. 9, no. 3, pp. 204–215, 2015.

[5] G. Moerkotte, T. Neumann, and G. Steidl, "Preventing bad plans by bounding the impact of cardinality estimation errors," *Proc. VLDB Endowment*, vol. 2, no. 1, pp. 982–993, 2009.

[6] L. Getoor, B. Taskar, and D. Koller, "Selectivity estimation using probabilistic models," in *Proc. 2001 ACM SIGMOD Int. Conf. Management of Data*, 2001.

[7] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, and A. Kemper, "Learned cardinalities: Estimating correlated joins with deep learning," in *Proc. Conf. Innovative Data Systems Research (CIDR)*, 2019.

[8] R. Marcus *et al.*, "Neo: A learned query optimizer," *Proc. VLDB Endowment*, vol. 12, no. 11, pp. 1705–1718, 2019.

[9] Y. Han, Z. Wu, Z. Yang, R. Zhu, and J. Zhou, "Are we ready for learned cardinality estimation?" *Proc. VLDB Endowment*, vol. 15, no. 9, pp. 1640–1654, 2022.

[10] Z. Wu, A. Shaikhha, R. Zhu, K. Zeng, Y. Han, and J. Zhou, "BayesCard: Revitalizing Bayesian frameworks for cardinality estimation," arXiv:2012.14743, 2020.

[11] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM Computing Surveys*, vol. 51, no. 5, pp. 1–42, 2018.

[12] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016.

[13] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[14] R. Marcus and O. Papaemmanouil, "Deep reinforcement learning for join order optimization," in *Proc. 2nd Workshop on Data Management for End-to-End Machine Learning*, 2018, pp. 1–4.